Temos duas "teorias" sobre o mesmo domínio: a teoria informal (regras das linguistas) e a teoria operacional (regras implementadas), e uma evidência empírica (exemplos anotados) que mostra que elas divergem. Isso é exatamente o problema clássico de **revisão de teorias** em IA simbólica: revisar uma base de regras existente à luz de exemplos positivos e negativos, minimizando a mudança na teoria original. Vamos tentar ancorar o pipeline nessa literatura porque ela já resolve formalmente "como localizar e corrigir a regra errada dado um conjunto de exemplos que falham e que não falham".

O pipeline tem 5 etapas. As três primeiras são *diagnóstico* (onde e por que diverge); as duas últimas são *revisão* (como corrigir sem quebrar o que já funciona).

1. Alinhar as saídas em unidade comparável

Antes de comparar regras, precisamos comparar *as saídas* no mesmo espaço.

Pergunta: A anotação das linguistas e saída do extrator têm granularidade idêntica de span? Podemos usar o alinhamento da OIE2016 (matching por sobreposição de tokens em REL e em cada ARG, não igualdade exata de string), e classificar cada exemplo em:

- **match exato** (regra implementada = anotação)
- **match parcial** (fronteira errada)
- **falso positivo estrutural** (relação/argumento que não deveria existir)
- **falso negativo** (relação/argumento que faltou)
- outros?

Para separar "erros de fronteira de span" (provavelmente causados por regras de MWE, elipse, coordenação) de "erros de existência" (regra de disparo/condição, não de composição).

2. Instrumentar proveniência de regra (rule provenance)

Para cada exemplo que falha, precisamos saber **qual regra implementada disparou** e produziu aquele resultado.. Se o motor de extração for GREW ou algo baseado em reescrita de grafo, isso é direto: GREW consegue logar, por aplicação, qual regra do `.grs` disparou em qual nó/aresta (a estratégia de aplicação já é registrável). Se o sistema for outro (regras hand-coded em Python, CRF com features, etc.), precisamos adicionar esse logging manualmente para não virar tentativa e erro.

Resultado dessa etapa: uma tabela `exemplo → regra(s) disparada(s) → output → gold → tipo de divergência`.

3. Formalizar a regra da linguista no mesmo esquema da regra implementada

Aqui não é reescrever o sistema, mas produzir uma "regra-alvo" comparável estruturalmente à "regra-atual", para poder fazer *diff estrutural* entre as duas, não apenas diff comportamental via exemplos.

Isso daria, para cada regra, um par `(R_impl, R_gold)` no mesmo formalismo e depois o diff é sintático: qual condição do padrão está faltando, sobrando, ou com escopo errado (ex.: `R_impl` casa `mwe` só quando contíguo na superfície; `R_gold` exige herança via fecho transitivo da aresta `mwe`, independente de contiguidade ?).

4. Revisão guiada por contraexemplo (CEGAR-style? ILP-style?)

Agora vamos tratar isso busca no espaço de revisões de `R_impl`, com dois conjuntos de restrição:

- **Exemplos que falharam (E^-)**: a regra revisada precisa passar a cobri-los corretamente.
- **Exemplos que não falharam (E^+)**: a regra revisada não pode deixar de cobri-los

1. proponha revisão candidata de `R_impl` (guiada pelo diff estrutural do passo 3);
2. rode contra todo $E^+ \cup E^-$;
3. se E^- passa a validar e E^+ continua validando → aceita;
4. se E^+ quebra → o contraexemplo específico que quebrou vira restrição adicional, refina-se a condição do padrão (tipicamente adicionando uma condição de exceção — `without {...}` no GREW, ou uma restrição de integridade no ASP);
5. repete.

Se tivermoa `R_impl` e `R_gold` representadas como fatos Datalog/ASP e E^+/E^- como instâncias concretas de grafo, dá para formular a busca da regra revisada como **problema de satisfação** : clingo encontra a revisão mínima (menor distância de edição na condição do padrão) que satisfaz todos os exemplos simultaneamente, em vez de você editar manualmente e re-rodar a cada tentativa.

5. Fechar o loop com as linguistas (verbalização)

Depois de cada revisão aceita, gere automaticamente uma **verbalização em linguagem controlada** da regra revisada (o inverso do passo 3: padrão formal → texto), e submeta às linguistas para confirmação.

1. Representação da instância (um exemplo = um grafo de dependência anotado)

Cada exemplo anotado vira um conjunto de fatos. Suponha o exemplo problemático: "de acordo com o relatório", com "de acordo com" como MWE preposicional descontínua na anotação gold, mas a `R_impl` atual só casa MWE contígua.

prolog

```
% ---- exemplo e1 (um dos que falharam, ∈ E⁻) ----

word(e1, w1, "de").

word(e1, w2, "acordo").

word(e1, w3, "com").

word(e1, w4, "relatorio").


% aresta mwe liga o núcleo (w2) aos satélites (w1, w3) — mesmo não-contíguos linearmente

mwe_edge(e1, w2, w1, prep).

mwe_edge(e1, w2, w3, prep).


% role0: atribuição feita por OUTRAS regras antes da regra de MWE entrar em ação

role0(e1, w2, rel).      % núcleo "acordo" já marcado REL por outra regra

role0(e1, w4, arg).      % "relatorio" já marcado ARG


% gold (anotação da linguista) — o que TODO nó do MWE deveria receber
```

```
gold(e1, w1, rel).

gold(e1, w2, rel).

gold(e1, w3, rel).

gold(e1, w4, arg).

prolog

% ---- exemplo e2 (um dos que NÃO falharam, ∈ E⁺ — não pode regredir) ----

word(e2, w1, "no").

word(e2, w2, "entanto").

word(e2, w3, "ele").

word(e2, w4, "saiu").


mwe_edge(e2, w1, w2, adv).   % "no entanto" — MWE contígua, já funciona hoje


role0(e2, w1, rel).

role0(e2, w3, arg).

role0(e2, w4, rel).


gold(e2, w1, rel).

gold(e2, w2, rel).

gold(e2, w3, arg).

gold(e2, w4, rel).

prolog
```

```
% ---- exemplo e3 (∈ E⁺, o caso que a MUDANÇA INGÊNUA quebraria) ----

% duas MWEs distintas, adjacentes no grafo mas semanticamente separadas

word(e3, w1, "apesar").

word(e3, w2, "disso").

word(e3, w3, "além").

word(e3, w4, "disso").


mwe_edge(e3, w1, w2, conj).   % "apesar disso"

mwe_edge(e3, w3, w4, conj).   % "além disso" — MWE DIFERENTE, não deve se fundir com a anterior


role0(e3, w1, rel).

role0(e3, w3, arg).          % núcleos com papéis DIFERENTES


gold(e3, w1, rel). gold(e3, w2, rel).

gold(e3, w3, arg). gold(e3, w4, arg).
```

`R_impl` atual (baseline, para diagnóstico) e o espaço de revisão

A regra atual, hoje, provavelmente é algo equivalente a:

prolog

```
% R_impl (hipótese: exige contiguidade linear w2 = w1+1, o bug suspeito)

role(E, W2, R) :- mwe_edge(E, W1, W2, _), role0(E, W1, R),
```

word_index(E, W1, I1), word_index(E, W2, I2), I2 = I1+1.

Isso falha em e1 (w3 não é adjacente a w2), exatamente o contraexemplo de E^-.

Em vez de corrigir isso "na mão", parametrizamos as condições candidatas como flags que o solver escolhe, e deixamos ele achar a combinação mínima que satisfaz $E^- \cup E^+$ simultaneamente:

prolog

```
% ---- espaço de hipóteses: cada condição é opcional, o solver escolhe incluir ou não ----

#const max_cond = 4.

cond(contig).       % exige adjacência linear (condição original, possivelmente removível)

cond(same_type).    % exige que W1,W2 compartilhem o mesmo tipo de mwe_edge

cond(no_prior_role). % exige que W2 ainda não tenha role0 próprio (evita fundir MWEs distintas)

cond(transitive).   % permite fecho transitivo da relação mwe (não só 1 hop)


% escolha: cada condição do espaço de hipótese é incluída (1) ou não (0) na regra revisada

{ include(C) } :- cond(C).


% ---- regra revisada, parametrizada pelas condições ativas ----

candidate_role(E, W2, R) :-

    mwe_edge(E, W1, W2, T1), role0(E, W1, R),

    word_index(E,W1,I1), word_index(E,W2,I2),
```

```
    ( not include(contig) ; I2 = I1+1 ),

    ( not include(same_type) ; mwe_edge(E, W1, W2, T1) ),   % type já fixo aqui por simplicidade

    ( not include(no_prior_role) ; not role0(E, W2, _) ).


% papel final = role0 original, OU o herdado pela regra candidata (sem conflito)

role(E, W, R) :- role0(E, W, R).

role(E, W, R) :- candidate_role(E, W, R).


% ---- integridade: nenhum nó pode ter dois papéis diferentes simultaneamente ----

:- role(E, W, rel), role(E, W, arg).

prolog

% ---- restrições rígidas vindas dos exemplos (isto é o "CEGAR" propriamente dito) ----


% TODO exemplo de E⁻ precisa bater com o gold em TODO nó

:- gold(E,W,R), not role(E,W,R), example_type(E, negativo).


% TODO exemplo de E⁺ precisa CONTINUAR batendo (restrição de não-regressão)

:- gold(E,W,R), not role(E,W,R), example_type(E, positivo).


% e nenhum papel espúrio pode aparecer onde o gold não previu

:- role(E,W,R), not gold(E,W,R).
```

```
example_type(e1, negativo).

example_type(e2, positivo).

example_type(e3, positivo).
```

prolog

```
% ---- minimalidade: prefira a revisão com MENOS condições alteradas em relação a R_impl ----

r_impl_conditions(contig).   % única condição presente na regra atual


edit(C) :- cond(C), include(C), not r_impl_conditions(C).      % condição nova adicionada

edit(C) :- cond(C), not include(C), r_impl_conditions(C).      % condição removida


#minimize { 1,C : edit(C) }.
```

Rodando isso no `clingo`, o solver busca, dentre as 2^4 combinações de condições, a única (ou o conjunto mínimo) que satisfaz `e1` (gold) e não quebra `e2`/`e3`. Nesse desenho de exemplo, a resposta esperada seria: remover `contig`, adicionar `no_prior_role`, encontrado por busca em vez de intuição.

3.ILASP

O encoding acima é ilustrativo e funciona em escala pequena, mas tem uma limitação séria: o espaço de hipóteses é fixo à mão (precisa antecipar `cond(contig)`, `cond(same_type)` etc.).

Com o ILASP declaramos um espaço de busca por mode declarations (que predicados/variáveis podem aparecer no corpo da regra aprendida) em vez de enumerar condições manualmente, e passa os exemplos

como interpretações parciais positivas/negativas. O solver aprende a regra (não só escolhe entre variantes pré-definidas) e já implementa otimalidade (regra mais simples que cobre os exemplos) e suporte a exemplos ruidosos.

prolog

```
% mode declarations — define o espaço de busca (não a regra em si)

#modeh(role(var(word), const(role))).

#modeb(mwe_edge(var(word), var(word), const(mwetype))).

#modeb(role0(var(word), const(role))).

#modeb(not role0(var(word), const(role))).

#modeb(word_index(var(word), var(int))).


% background knowledge = os fatos de grafo (word/2, mwe_edge/4, role0/3) de todos os exemplos

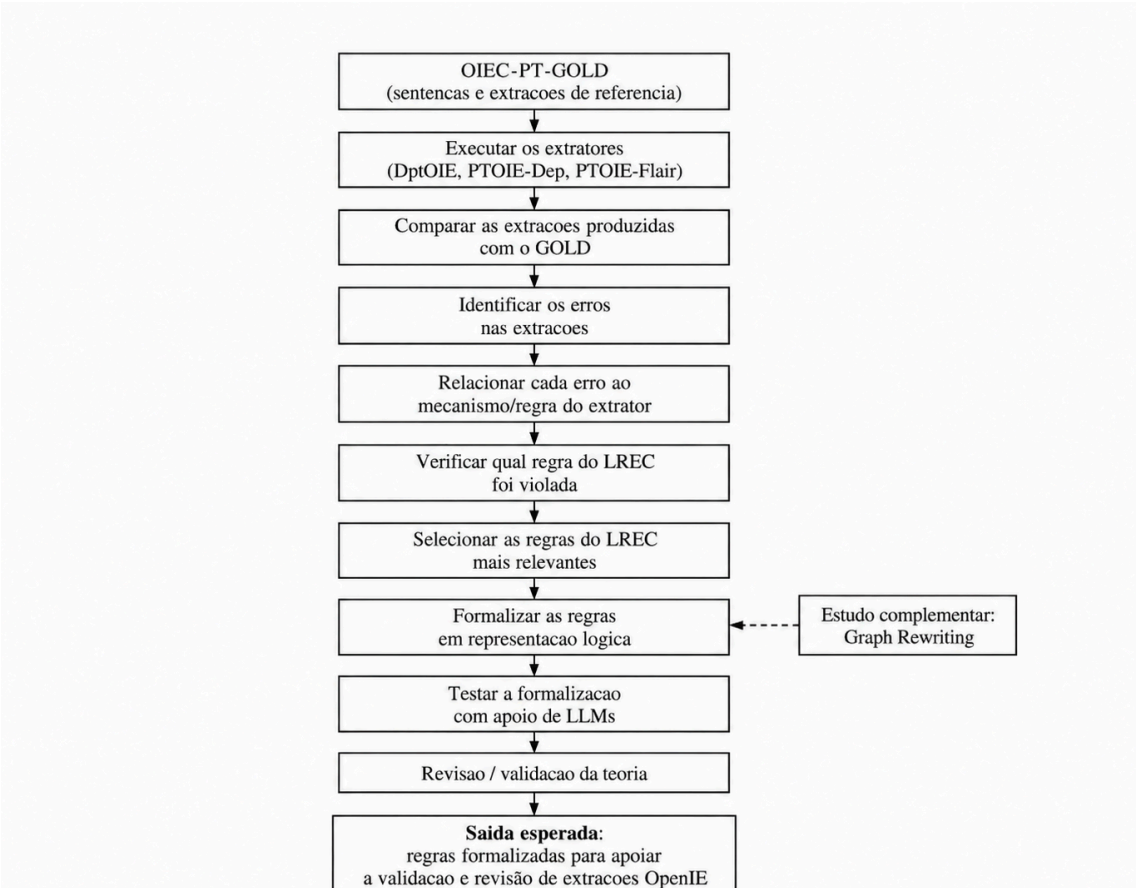

% exemplos como interpretações parciais (inclusão/exclusão de átomos + contexto do exemplo)

#pos(e1_ex, { role(w1,rel), role(w2,rel), role(w3,rel) }, {}, { mwe_edge(w2,w1,prep). mwe_edge(w2,w3,prep). role0(w2,rel). }).

#pos(e2_ex, { role(w1,rel), role(w2,rel) }, {}, { mwe_edge(w1,w2,adv). role0(w1,rel). }).

#neg(e3_ex, { role(w2, arg) }, {}, { mwe_edge(w1,w2,conj). mwe_edge(w3,w4,conj). role0(w1,rel). role0(w3,arg). }).
```

O ILASP devolve diretamente a regra ASP aprendida (não uma combinação de flags), já minimizada por tamanho de hipótese;com essa saída traduzimos de volta para GREW


OIEC-PT-GOLD
(sentencas e extracoes de referencia)
Executar os extratores
(DptOIE, PTOIE-Dep, PTOIE-Flair)
Comparar as extracoes produzidas
com o GOLD
Identificar os erros
nas extracoes
Relacionar cada erro ao
mecanismo/regra do extrator
Verificar qual regra do LREC
foi violada
Selecionar as regras do LREC
mais relevantes
Formalizar as regras
em representacao logica
Estudo complementar:
Graph Rewriting
Testar a formalizacao
com apoio de LLMs
Revisao / validacao da teoria
Saida esperada:
regras formalizadas para apoiar
a validacao e revisão de extracoes OpenIE